

</>



Dezyne E'cole College
Business • Design • Computers

Data
structure
& C++

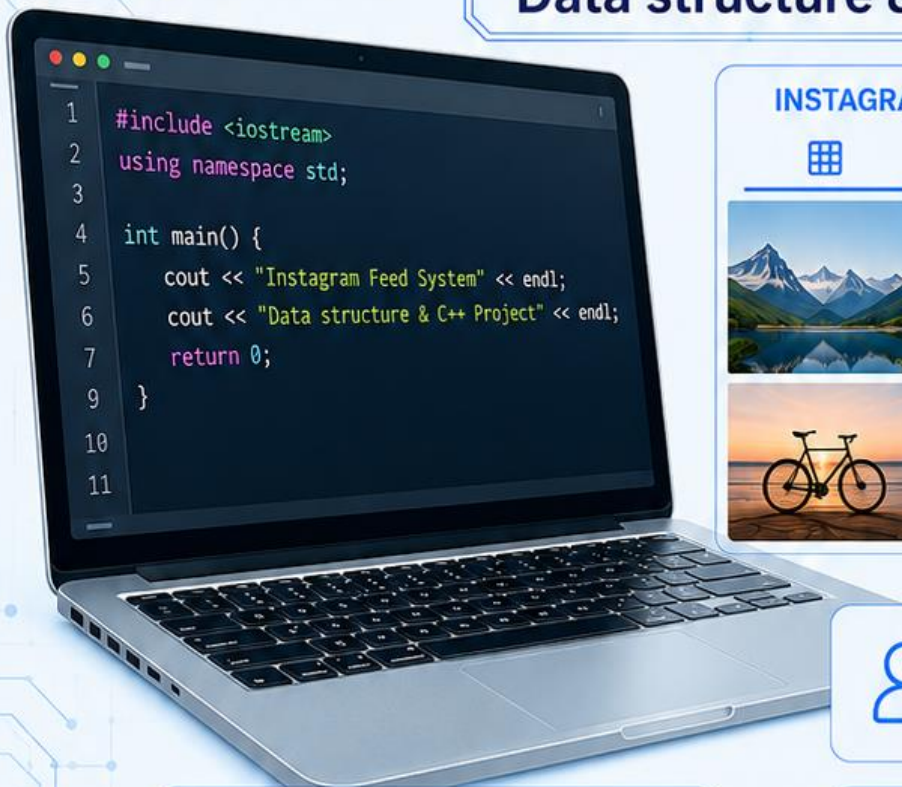
</>

{ }

>_

INSTAGRAM FEED SYSTEM MANAGEMENT

Data structure & C++



INSTAGRAM FEED PREVIEW



USER
MANAGEMENT



POST
MANAGEMENT

FEATURES

- Insert Post
- Delete Post
- Show Post
- Search Post
- Count Post



STUDENT NAME

Jaishree Gahlot



CLASS & SECTION

BCA 1st year

Project Report on
Instagram Feed System
Management using Link
List

Submitted to

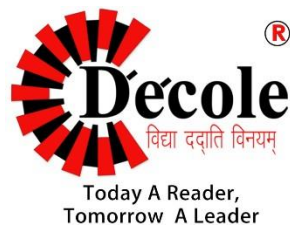
Dezyne École College

Towards the Partial Fulfilment of

BCA 2nd Semester

By

Jaishree Gahlot



www.dezyneecole.com 2025-

2026

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,

Jaishree Gahlot

Department of BCA

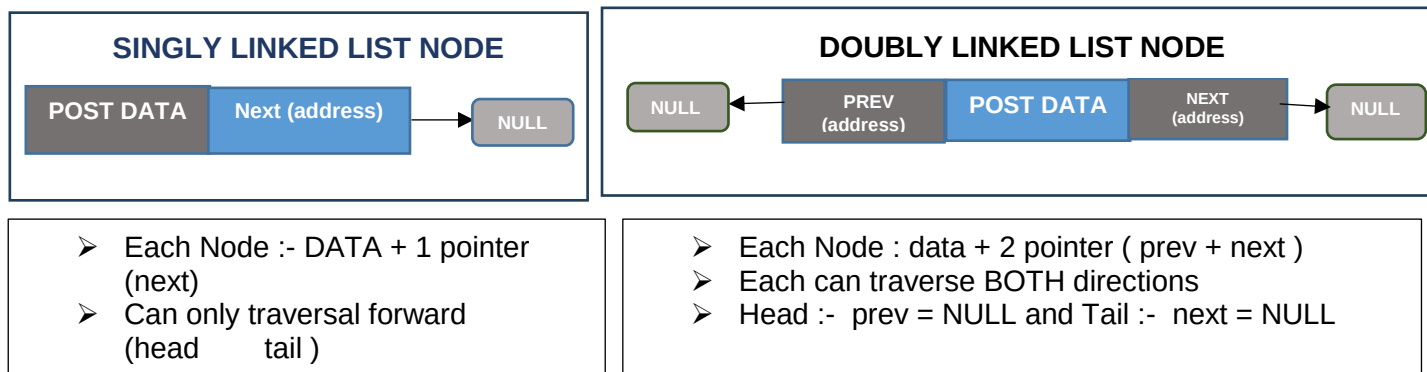
Dezyne École College, Ajmer

Part 1: Introduction — What is a Linked List?	1
Singly vs Doubly — At a Glance	2
Why Not an Array?	2
Insert at Head — $O(1)$	4
Insert at Tail — $O(1)$ with tail pointer	6
Insert at Position k — $O(k)$	7
Delete Operations — 3 Cases	9
Traversal — $O(n)$	13
Search by Keyword — $O(n)$	14
Count Nodes — $O(1)$ Best Practice	15
Part 2: Doubly Linked List — Insertion, Deletion & Traversal	16
Insert at Tail — $O(1)$	20
Insert at Position k — $O(k)$	21
Delete Head — $O(1)$	21
Delete Tail — $O(1)$ ★ Key Advantage Over Singly	24
Delete given position Node — $O(1)$ with pointer / $O(n)$ if search required	25
Forward & Backward Traversal — $O(n)$ each	27

Part 1: Introduction — What is a Linked List?

A Linked List is a linear data structure where elements (called nodes) are NOT stored in contiguous memory locations. Each node holds its data and one or two pointers — memory addresses — that link it to neighbouring nodes. The nodes can live anywhere in RAM; the pointers stitch them into a logical sequence.

Real World Context of - INSTAGRAM FEED SYSTEM



Instagram Post timeline shows Posts in reverse-chronological order — newest at the top, oldest at the bottom. Three operations dominate this system:

- **Insert at head:** A new Post always goes at the very top. This must be instant — $O(1)$.
- **Delete from any position:** Content moderation removes any Post immediately — $O(1)$.
- **Traverse from head to tail:** Displaying the timeline scrolls from newest to oldest — $O(n)$.

These three needs define the **Linked List** — a data structure that does all three efficiently.

Real-World Context — Instagram Timeline

Instagram Post timeline shows Posts in reverse-chronological order — newest at the top, oldest at the bottom. Three operations dominate this system:

- **Insert at head:** A new Post always goes at the very top. This must be instant — $O(1)$.
- **Delete from any position:** Content moderation removes any Post immediately — $O(1)$.
- **Traverse from head to tail:** Displaying the timeline scrolls from newest to oldest — $O(n)$.

These three needs define the **Linked List** — a data structure that does all three efficiently.

Singly vs Doubly — At a Glance

Property	Singly Linked List	Doubly Linked List
Pointers per node	1 (NEXT only)	2 (PREV + NEXT)
Traversal direction	Forward only (head → tail)	Both directions
Delete tail	$O(n)$ — must scan to 2nd-last	$O(1)$ — use tail.prev
Delete any node (with ptr)	$O(n)$ — scan for predecessor	$O(1)$ — both neighbours known
Extra memory	Less (1 ptr × n)	More (2 ptr × n = +8 bytes/node)
Typical use	Stacks, hash chaining, simple queues	Timelines, browsers, LRU cache, dequeues

Why Not an Array?

Operation	Array	Linked List	Instagram Need
Insert at front	$O(n)$ — shift all elements right	$O(1)$ — rewire 2 pointers	New Song on top — must be instant
Delete any element	$O(n)$ — shift remaining elements	$O(1)$ — rewire neighbours	Remove any Song instantly
Access by index	$O(1)$ — direct formula	$O(n)$ — traverse from head	Not needed for sequential display
Dynamic size (grow)	Needs reallocation	Trivial — just add a node	Song count is unpredictable
Search	$O(n)$ linear, $O(\log n)$ binary	$O(n)$ linear only	Both require traversal for unsorted data

Memory Layout Example:

public:

string instagramid;

string UNAME;

```

string instagram;
NODE *NEXT;
NODE(string x, string y, string z)
{
    instragramid = x;
    UNAME = y;
    instagram = z;
    NEXT = nullptr;
}
};
class LinkList
{
public:
    string instragramid;
    int ICNT;
    string UName, Uinstagram;
    NODE *START,*END;
    int countNode;
    LinkList(){
        ICNT = 1001;
        countNode = 0;
        START = nullptr;
        END = nullptr;
    }
}

```

Insert at Head — $O(1)$

Singly linked list – Insert at Head & Insert at Tail

INSERT AT HEAD

BEFORE :-



1. New node = new node (Post A)



INSERT AT TAIL — $O(1)$ WITH TAIL POINTER

BEFORE :-



1. New node = new node (song C)
2. `tail.next = head (= song B)`
3. `tail = newNode`

AFTER :-



INSERT AT GIVEN POSITION

BEFORE :-



1. `Cur = head` :- travers 1 step to reach pos 1 (POST B)
2. `NewNode.next = cur.next (= POST C at pos 2)`
3. `Cur.next = newNode`

Concept

The newest Post always goes at the top of the timeline. Head insertion achieves this in exactly 2 pointer operations — always $O(1)$ regardless of how many Songs already exist

Step-by-Step Algorithm

1. **Create new node:** `PostNode* newNode = new PostNode(text, user);` — allocates heap memory, sets `next = NULL`.
2. **Connect forward:** `newNode->next = head;` — new node now points to the current head (will become 2nd node).
3. **Move head pointer:** `head = newNode;` — head now refers to the new node.
4. **Handle empty list:** If the list was empty, also set `tail = newNode;`
5. **Increment counter:** `size++;`

Code:

```
int insertNode(int opr)
{
    system("cls");
    if (opr == 1){
        cout<<"\n\nInsert At Beginning . \n\n";
        getPost();
        NODE *TEMP = new NODE(instragramid, UName, Uinstagram);
        if (TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
            return 0;
        }
        if (START == nullptr){
            END = START = TEMP;
        }
        else{
            TEMP->NEXT = START;
            START = TEMP;
        }
        countNode++;
        ICNT++;
    }
}
```

Insert at Tail — $O(1)$ with tail pointer

Without tail pointer: Must traverse all n nodes to reach the last one, then append. Cost = $O(n)$.

With tail pointer (always recommended): Directly access tail, attach new node, advance tail. Cost = $O(1)$.

Step-by-Step Algorithm (with tail pointer)

1. **Create new node** `PostNode* newNode = new PostNode(text, user);` — `newNode->next = NULL`.
2. **Handle empty list:** If `head == NULL`: `head = tail = newNode;` return.
3. **Connect:** `tail->next = newNode;` — current tail now points to the new node.
4. **Advance tail:** `tail = newNode;` — tail pointer moves to the new last node.
5. **Increment counter:** `size++;`

Code

```
else if (opr == 2){
    cout<<"\n\nInsert At Post End . \n\n";
    getPost();
    NODE *TEMP = new NODE(instragramid,UName, Uinstagram);
    if (TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    if (START == nullptr){
        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        END = TEMP;
    }
    countNode++;
}
```

```

    ICNT++;
}

```

Insert at Position k — $O(k)$

Three sub-cases

- $k = 0$: Same as `insertAtHead()` — $O(1)$.
- $k = \text{size (last position + 1)}$: Same as `insertAtTail()` — $O(1)$.
- $0 < k < \text{size (middle)}$: Traverse $k-1$ steps to find the predecessor node, then rewire 2 pointers.

Step-by-Step Algorithm for Middle Insert

1. Create `newNode`.
2. Start `curr = head`; traverse $k-1$ steps: `for(int i=0; i<k-1; i++) curr = curr->next;`
3. `newNode->next = curr->next;` — new node points to what `curr` was pointing to.
4. `curr->next = newNode;` — `curr` now points to new node.
5. `size++;`

Code :

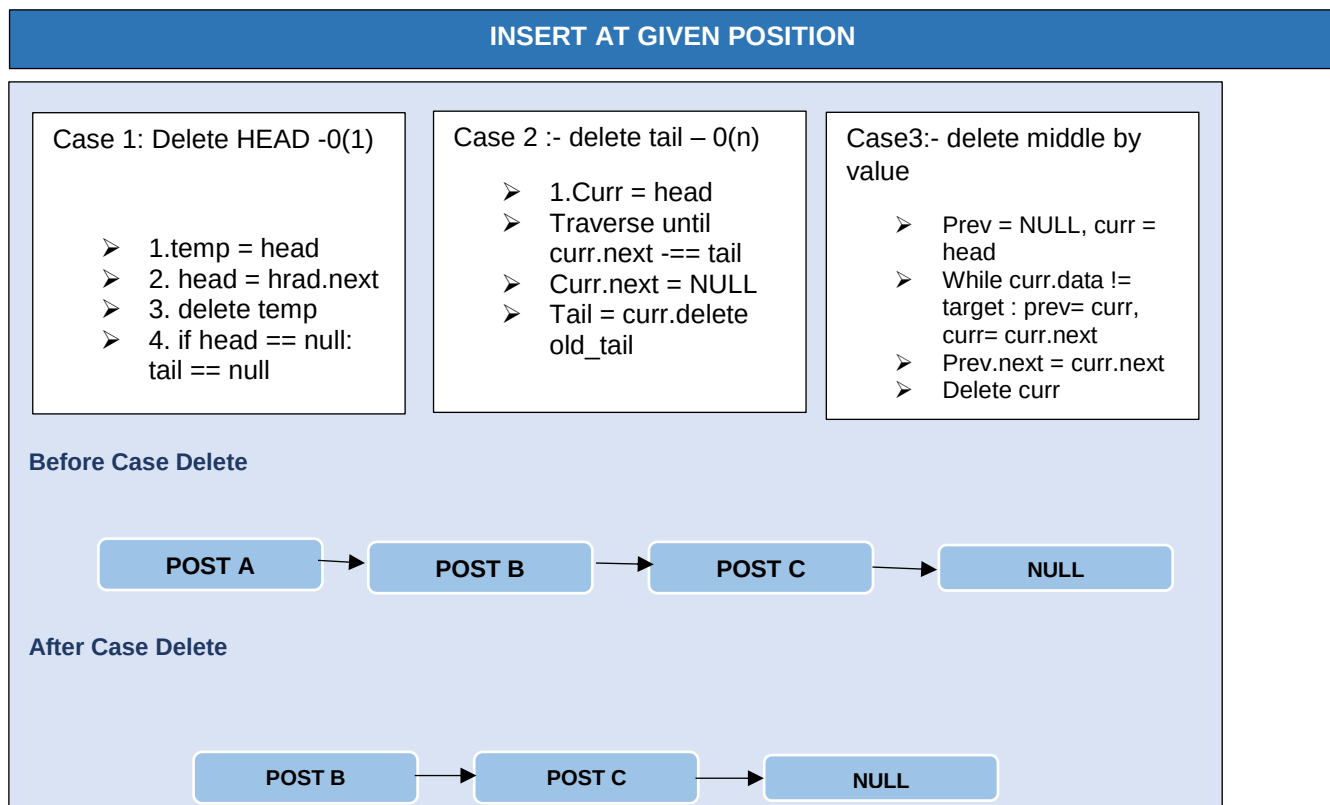
```

else if ( opr == 3){
    cout<<"\n\nInsert Post At Given Position . \n\n";
    cout<<"Total Post "<<"\n\n";
    int pos;
    getPost();
    cout<<"Enter Position Between 0 to "<<countNode<<" -- ";
    cin>>pos;

    NODE *TEMP = new NODE(instragramid,UName, Uinstagram);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODE *TEMP1 = START;
    for ( i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
}

```

```
if ( START == nullptr){
    END = START = TEMP;
}
else{
    TEMP->NEXT = TEMP1->NEXT;
    TEMP1->NEXT = TEMP;
    if ( TEMP1 == END){
        END = TEMP1->NEXT;
    }
}
countNode++;
ICNT++;
}
```



Case 1 — Delete Head — O(1)

Moving the head pointer one step forward effectively removes the first node. No traversal needed.

1. **temp = head;** — save the node to free later.
2. **head = head->next;** — head now points to the 2nd node (new head).
3. **If head != NULL:** it was a single-node list: set **tail = NULL;**
4. **delete temp;** — free heap memory.
5. **size--;**

```
int deleteNode(int opr)
{
    NODE *TEMP;
    if (opr == 1){
        cout<<"\n\nDelete From Beginning . \n\n";
        if (START == nullptr)
        {
            cout<<"Post is Empty.. \n\n";
            return 0;
        }
    }
```

```

    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        START = START->NEXT;
    }
    countNode--;
    delete(TEMP);
    cout<<"\nPost Deleted Successfully\n\n";
}

```

Case 2 — Delete Tail — $O(n)$ ← Singly's Weakness

Problem: To delete the tail we must update the second-to-last node's next pointer to NULL. In a singly linked list the only way to find that node is to traverse from head — $O(n)$. This is why doubly linked lists are preferred when tail deletion is frequent.

1. Start **curr = head**; traverse until **curr->next == tail**;
2. **curr->next = NULL**; — second-to-last now terminates the list.
3. **delete tail**; then **tail = curr**;
4. **size--**;

```

else if ( opr == 2){
    cout<<"\n\nDelete From End . \n\n";
    if ( START == nullptr)
    {
        cout<<"Post is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        int c = 1;
        while ( TEMP->NEXT != END)
        {
            cout<<c++<<endl;
            TEMP = TEMP->NEXT;
        }
    }
}

```

```

    }
    NODE *T1 = END;
    TEMP->NEXT = nullptr;
    END = TEMP;
    cout<<END->UNAME<<endl;
    delete(T1);
}
countNode--;
cout<<"\nPost Deleted Successfully\n\n";
}

```

Case 3 — Delete Middle Node by Given Position — $O(n)$

Find the target node and its predecessor simultaneously using two pointers, then bypass the target.

1. **prev = NULL; curr = head;**
2. **Traverse: while curr != NULL and curr->data != target: prev = curr; curr = curr->next;**
3. **If curr == NULL: value not found — return.**
4. **prev->next = curr->next; — predecessor skips over the target node.**
5. **If curr == tail: tail = prev; — update tail pointer.**
6. **delete curr; size--;**

```

else if ( opr == 3){
    cout<<"\n\nDelete From Given Position . \n\n";

    if ( START == nullptr)
    {
        cout<<"Post is Empty.. \n\n";
        return 0;
    }
    int pos;
    cout<<"Enter Position for Delete (1 - "<<countNode<<") - ";
    cin>>pos;
    if ( pos == 1 && START == END)
    {
        TEMP = START;
        START = END = nullptr;
    }
    else{
        TEMP = START;
        for (int i = 2 ; i < pos ; i++)
        {

```

```

        TEMP = TEMP->NEXT;
    }
    if (TEMP->NEXT == END){
        NODE *T1 = END;
        TEMP->NEXT = nullptr;
        END = TEMP;
    }
    else{
        NODE *T1 = TEMP->NEXT;
        TEMP->NEXT = T1->NEXT;
        delete(T1);
    }
}
countNode--;
cout<<"\nPost Deleted Successfully\n\n";
}

```


Singly Linked List – Traversal , Search , Count & Reverse

Traversal – $O(n)$

- I. Start curr = head
- II. Loop : while curr != null
 - a. Print curr.data
 - b. Curr = curr.next
- III. End :- print 'NULL'

Search – $O(n)$ always

- I. Curr = head.scanned = 0
- II. While curr != NULL;
- III. Scanned ++
- IV. If curr.data has keyword
- V. Return curr <- FOUND

COUNT – $O(1)$ best practice

- I. Count = 0 : curr = head
- II. While curr != NULL: count++

Method 2(maintain counter)

- I. Class has count size = 0
- II. Insert node(): size++
- III. deleteNode () size --

Reverse – $O(n)$, $O(1)$ space

- I. Prev = NULL
- II. While curr != NULL
- III. Curr = curr.next <- save
- IV. Curr.next = prev <- filp
- V. Prev = curr

Concept

Traversal visits every node exactly once, following next pointers from head until NULL. This is how Instagram renders your timeline from top to bottom — each Post displayed in order.

1. **curr = head;** — start at newest Post.
2. Loop: **while (curr != nullptr)** — stop when we fall off the end.
3. Inside loop: process curr->data; then **curr = curr->next;**

```

void showNode()
{
    cout<<"\n\ninstagram Post -- \n\n";
    cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"null \n\nPost is empty\n";
    }
    else{
        while ( TEMP!= nullptr){
            cout<<"["<<TEMP->instragramid<<" ";
            cout<<TEMP->UNAME<<" ";
            cout<<TEMP->instagram<<"] -> ";
            TEMP = TEMP->NEXT;
        }
        cout<<"Null";
    }
}

```

Search by Keyword — $O(n)$

Why $O(n)$ only: Linked lists have no index formula. To find a node you must start at head and examine each node one by one. Best case $O(1)$ if found at head. Worst case $O(n)$ if at tail or absent.

Why binary search is impossible: Binary search requires jumping to the middle element. In a linked list, reaching node i requires following i pointers from head — $O(i)$. So finding the middle costs $O(n/2)$, making binary search $O(n \log n)$ — worse than linear $O(n)$.

```

int searchinstagramlnNode()
{
    cout<<"\n\nSearch Instagram Post in Nodes -- \n\n";
    cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"null \n\nlist is empty\n";
        return 0;
    }
    else{
        cout<<"\n\nEnter User Name To Find instagrams -- ";
        string uname;

```

```

int counter = 0;
getline(cin,uname);
while ( TEMP!= nullptr){
    if (TEMP->UNAME == uname)
    {
        cout<<"\n["<<TEMP->instragramid<<" , ";
        cout<<TEMP->UNAME<<" , ";
        cout<<TEMP->instagram<<"]\n";
        counter++;
    }
    TEMP = TEMP->NEXT;
}
if ( counter == 0){
    cout<<"\nNo instagram Found..\n\n";
}
}

```

Count Nodes — O(1) Best Practice

Approach	Algorithm	Time	Recommendation
Traverse and count	curr=head; while curr≠NULL: count++; curr=curr.next	O(n) per call	Avoid for frequent calls
Maintain size counter	size++ on insert, size-- on delete, getCount() returns size	O(1) per call	Always use this approach

```

void countNodes()
{
    system("cls");
    cout<<"Count instagram Module --\n\n";
    cout<<"Total Number of Post -- "<<countNode<<" Post Available\n\n";
}

```

Part 2: Doubly Linked List — Insertion, Deletion & Traversal

What the prev Pointer Adds

A doubly linked list adds one extra pointer — prev — to every node. This single addition costs 8 bytes per node on a 64-bit system but unlocks three major advantages:

- **O(1) delete of any node** — both neighbours are immediately accessible via prev and next. No scanning needed.
- **O(1) tail deletion** — tail.prev gives the second-to-last node directly. No traversal.
- **Backward traversal** — start at tail, follow prev pointers. Impossible in singly linked lists.

C++ Node Structure for Doubly Linked List

```
class NODED
```

```
{
```

```
public:
```

```
    NODED *PREV;
```

```
    string ID;
```

```
    string UNAME;
```

```
    string Instagram ;
```

```
    NODED *NEXT;
```

```
    NODED(string x, string y, string z)
```

```
{
```

```
    PREV = nullptr;
```

```
    ID = x;
```

```
    UNAME = y;
```

```
    Instagram = z;
```

```
    NEXT = nullptr;
```

```
}
```

```
};
```

```
class DLinkedList
```

```
{
```

```
public:
```

```
    string instragramid;
```

```
    int ICNT;
```

```

string UName, UInstagram ;
NODED *START,*END;
int countNode;
DLinkedList(){
    ICNT = 1001;
    countNode = 0;
    START = nullptr;
    END = nullptr;
}
void getPost()
{
    instragramid = "Instagram Post"+ to_string(ICNT);
    cout<<"\nEnter Instragram Post id -- "<<instragramid<<endl;
    //getline(cin,instragramid);
    cout<<"Enter Username -- ";
    //cin.ignore();
    getline(cin,UName);
    cout<<"Enter Post Caption -- ";
    getline(cin,UInstagram );
}

```

INSERT AT HEAD

BEFORE :



AFTER:

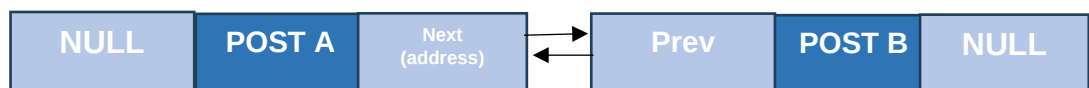


Steps :-

- 1 . create newNode
2. newNode.next = head(B)
- 3 . head. Prev = newNode <- key
4. head = newNode

INSERT AT END

BEFORE:



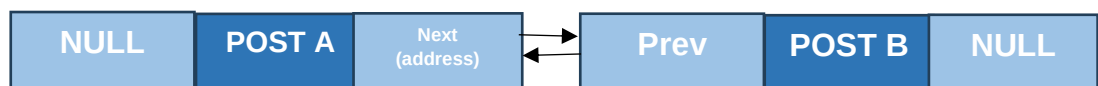
AFTER:



1. Create newNode
2. newNode prev = tail(B) <- key !
3. tail.next = newNode
4. tail = newNode

INSERT AT GIVEN POSTION

BEFORE :-



AFTER:-



Step

1. `newNode.prev = A`
2. `newNode.next = B`
3. `A.next = newNode`
4. `B.prev = newNode` <- doubly needs this extra

Extra Step vs Singly

Compared to singly insertion at head, doubly adds one step: setting the old head's prev pointer back to the new node.

Algorithm — 4 pointer operations

1. **Create:** `PostNodeDL* newNode = new PostNodeDL(text, user);` — `prev=NULL, next=NULL.`
2. **If empty:** `head = tail = newNode;` return.
3. **Connect forward:** `newNode->next = head;` — new node points to old head.
4. **Connect backward:** `head->prev = newNode;` — old head points BACK to new node.
← extra step vs singly!
5. **Advance head:** `head = newNode;`
6. `size++;`

```
int insertNode(int opr)
{
    system("cls");
    if (opr == 1){
        cout<<"\n\nInsert At Beginning . \n\n";
        getPost();
        NODED *TEMP = new NODED(instragramid,UName, UInstagram);
        if (TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
            return 0;
        }
        if (START == nullptr){
            END = START = TEMP;
        }
        else{
            TEMP->NEXT = START;
            START->PREV = TEMP;
            START = TEMP;
        }
        cout<<END->UNAME;
```

```

        countNode++;
        ICNT++;
    }

```

Insert at Tail — $O(1)$

Always $O(1)$: Doubly linked lists always maintain a tail pointer. Appending is direct — no traversal required in either the empty or non-empty case.

Algorithm — 4 pointer operations

1. **Create:** `PostNodeDL* newNode = new PostNodeDL(text, user);`
2. **If empty:** `head = tail = newNode;` return.
3. **Connect backward:** `newNode->prev = tail;` — new node looks BACK at current tail.
4. **Connect forward:** `tail->next = newNode;` — current tail points forward to new node.
5. **Advance tail:** `tail = newNode;`
6. `size++;`

```

else if ( opr == 2){
    cout<<"\n\nInsert At End . \n\n";
    getPost();
    NODED *TEMP = new NODED(instragramid,UName, UInstagram );
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
        END->NEXT = TEMP;
        TEMP->PREV = END;
        END = TEMP;
    }
    countNode++;
    ICNT++;
}

```


Insert at Position k — $O(k)$

Middle case — 4 pointer rewires (vs 2 in singly)

Traverse to position k-1 (same as singly), but now must wire FOUR pointers: both neighbours' prev and next must be updated.

```
else if ( opr == 3){
    cout<<"\n\nInsert At Given Position . \n\n";
    cout<<"Total Post -- "<<countNode<<"\n\n";
    int pos;
    getPost();
    cout<<"Enter Position Between 0 to "<<countNode<<" -- ";
    cin>>pos;

    NODED *TEMP = new NODED(instragramid,UName, UInstagram);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODED *TEMP1 = START;
    for ( i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{

        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        TEMP->PREV = TEMP1;
        TEMP->NEXT->PREV=TEMP;
        if ( TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    countNode++;
    ICNT++;
```

Delete Head — $O(1)$

DELETE AT HEAD

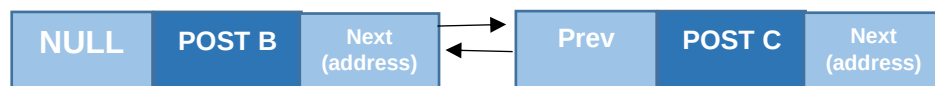
DELETE HEAD — $O(1)$

1. Head = head.next
2. Head.prev = NULL <- new
head has no pred
3. Delete old_head
4. If head == NULL tail = NULL

BEFORE:



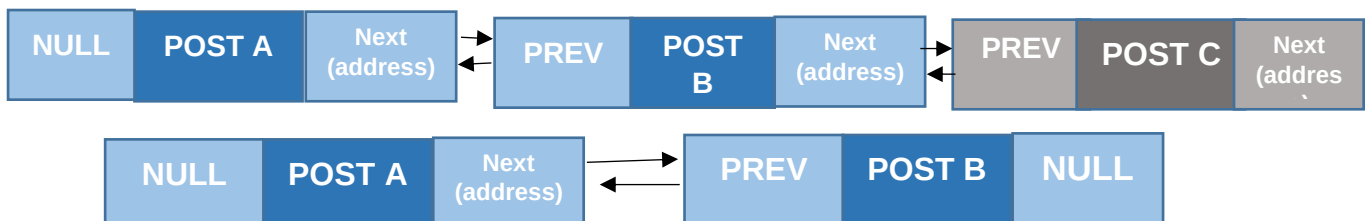
AFTER:



DELETE AT TELL

DELETE TAIL — $O(1)$

1. Tail = tail.orev <- via prev
pointer
2. Tail.next = NULL
3. Delete old_tail
4. If tail == NULL ,head = NULL



DELETE AT GIVEN POSTION

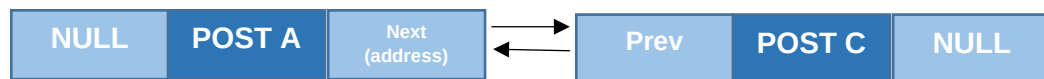
DELETE AT POSITION- 0(1)

1. $N.\text{prev}.\text{next} = n.\text{next}$ (A skip over N)
2. $N.\text{next}.\text{prev} = N.\text{prev}$ (B pointer back to A)
3. Delete N

Before



After



Algorithm

1. **Check:** If $\text{head} == \text{NULL}$: empty list, return
2. **Save:** $\text{SongNodeDL}^* \text{temp} = \text{head};$
3. **Advance head:** $\text{head} = \text{head} \rightarrow \text{next};$
4. If $\text{head} \neq \text{NULL}$: $\text{head} \rightarrow \text{prev} = \text{nullptr};$ — new head has no predecessor
5. Else (list now empty): $\text{tail} = \text{nullptr};$
6. **Free:** $\text{delete temp}; \text{size--};$

Delete node at head

```
int deleteNode(int opr)
```

```
{
    NODED *TEMP;
    if (opr == 1){
        cout<<"\n\nDelete From Beginning . \n\n";
        if (START == nullptr)
        {
```

```

        cout<<"Post is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        START = START->NEXT;
        START->PREV=nullptr;
    }
    countNode--;
    delete(TEMP);
    cout<<"\nPost Deleted Successfully\n\n";
}

```

Delete Tail — $O(1)$ ★ Key Advantage Over Singly

Why This is Doubly's Key Advantage

In a singly linked list, deleting the tail requires $O(n)$ traversal to find the second-to-last node (which must have its next set to NULL). In a doubly linked list, `tail.prev` ALREADY points to the second-to-last node — $O(1)$, no scanning!

Algorithm — 3 pointer operations

1. **Check:** If `tail == NULL`: empty list, return.
2. **Save:** `PostNodeDL* temp = tail;`
3. **Advance tail BACKWARD:** `tail = tail->prev;` ← ONLY possible because of the prev pointer!
4. If `tail != NULL`: `tail->next = nullptr;` — new tail has no successor.
5. Else (list now empty): `head = nullptr;`
6. `delete temp; size--;`

```

else if ( opr == 2){
    cout<<"\n\nDelete From End . \n\n";
    if ( START == nullptr)
    {

```

```

        cout<<"Post is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        NODED *TEMP = END;
        END = END->PREV;
        END->NEXT = nullptr;
        delete(TEMP);
    }
    countNode--;
    cout<<"\nPost Deleted Successfully\n\n";
}

```

Delete given position Node — $O(1)$ with pointer / $O(n)$ if search required

The Biggest Advantage

If you already have a direct pointer to the node to delete, $O(1)$ is achievable because both neighbours are accessible via prev and next. Just 2 rewires bypass the node completely.

Algorithm — 2 pointer rewires

1. Check edge cases: if $N == \text{head}$ → call `deleteHead()`; if $N == \text{tail}$ → call `deleteTail()`.
2. **A.next = N.next:** $N \rightarrow \text{prev} \rightarrow \text{next} = N \rightarrow \text{next}$; — predecessor skips over N.
3. **B.prev = N.prev:** $N \rightarrow \text{next} \rightarrow \text{prev} = N \rightarrow \text{prev}$; — successor points back past N.
4. **delete N; size--;**

```

else if ( opr == 3){

```

```

    cout<<"\n\nDelete From Given Position . \n\n";

```

```

    if ( START == nullptr)

```

```

    {

```

```

        cout<<"Post is Empty.. \n\n";

```

```

        return 0;

```

```

    }

```

```

    int pos;

```

```

    cout<<"Enter Position for Delete (1 - "<<countNode<<") - ";

```

```

cin>>pos;
if ( pos == 1 && START == END)
{
    TEMP = START;
    START = END = nullptr;
}
else{
    TEMP = START;
    for (int i = 1 ; i < pos ; i++)
    {
        TEMP = TEMP->NEXT;
    }
    if (TEMP == END){
        TEMP->PREV->NEXT = nullptr;
        END = TEMP->PREV;
        delete(TEMP);
    }
    else{
        TEMP->PREV->NEXT = TEMP->NEXT;
        TEMP->NEXT->PREV = TEMP->PREV;
        delete(TEMP);
    }
}
countNode--;
cout<<"\nPost Deleted Successfully\n\n";
}

```

Forward & Backward Traversal — $O(n)$ each

Doubly LinkList Forward And Backward Traversal



FORWARD TRAVERSAL

Step 1: Start
 Step 2: Set PTR = HEAD
 Step 3: If HEAD = NULL
 Print "List is Empty"
 Go to Step 7
 Step 4: Repeat while PTR ≠ NULL
 Print PTR->DATA
 PTR = PTR->NEXT
 Step 5: End Repeat
 Step 6: Stop Traversal

BACKWORD TRAVERSAL

Step 1: Start
 Step 2: Set PTR = TAIL
 Step 3: If TAIL = NULL
 Print "List is Empty"
 Go to Step 7
 Step 4: Repeat while PTR ≠ NULL

Forward Traversal (Newest → Oldest)

```

int showNode()
{
    cout<<"\n\nDoubly Link List Show Post -- \n\n";
    cout<<"* Head -> ";
    if ( START == nullptr){
        cout<<"null \n\nlist is empty\n";
        return 0;
    }
    cout<<"\n\n1.Show Forward Post ..\n";
    cout<<"2. Show Backward Post\n";
    cout<<"3. Goto to Main Menu..\n";
    cout<<"4. Exit\n";
    char c = getch();
  
```

```

if ( c == '1'){
    cout<<"\n\nShow Forward Post \n\n";
    cout<<"null <- ";
    NODED *TEMP = START;
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<" ";
        cout<<TEMP->UNAME<<" ";
        cout<<TEMP->Instagram<<"] -> ";
        TEMP = TEMP->NEXT;
    }
    cout<<"Null\n\n";
}

```

Backward Traversal (Oldest → Newest) ← Impossible in Singly

Unique to doubly linked lists. Start at tail, follow prev pointers. This direction is physically impossible in a singly linked list because there is no prev pointer.

```

else if ( c == '2'){
    cout<<"\n\nShow Backward Post\n\n";
    cout<<"null <- ";
    NODED *TEMP = END;
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<" ";
        cout<<TEMP->UNAME<<" ";
        cout<<TEMP->Instagram <<"] -> ";
        TEMP = TEMP->PREV;
    }
    cout<<"Null\n\n";
}
else if ( c == '3'){
    MainMenu();
}

```



```
}  
else if (c == '4'){  
    exit(0);  
}
```

Real-World Applications of Backward Traversal

Application	How Backward Traversal is Used
Browser back button	Navigate history in reverse (most recent → oldest visited)
Text editor undo/redo	Move backward through change history to undo actions
Music player Previous Track	Go backward through the playlist
LRU Cache eviction	Tail is LRU item — accessed via backward from end, evicted $O(1)$
Twitter / Social Media	Show thread in chronological order (oldest reply first)

Final code

Singly LinkList

```
#include<iostream>
#include<conio.h>
#include<string>
using namespace std;
class NODE
{
public:
    string instragramid;
    string UNAME;
    string instagram;
    NODE *NEXT;
    NODE(string x, string y, string z)
    {
        instragramid = x;
        UNAME = y;
        instagram = z;
        NEXT = nullptr;
    }
};
class LinkList
{
public:
    string instragramid;
    int ICNT;
    string UName, Uinstagram;
```

```

NODE *START,*END;

int countNode;

LinkedList(){
    ICNT = 1001;
    countNode = 0;
    START = nullptr;
    END = nullptr;
}

void getPost()
{
    instagramid = "instagram Post"+ to_string(ICNT);
    cout<<"\nEnter Post id -- "<<instagramid<<endl;
    //getline(cin,Tinstagramid);
    cout<<"Enter Username -- ";
    //cin.ignore();
    getline(cin,UName);
    cout<<"Enter Caption -- ";
    getline(cin,Uinstagram);
}

char continueCheck()
{
    char c;
    cout<<"\n\nAre you.. \n";
    cout<<"1. Continue with Same\n";
    cout<<"2. Go to Main Previous Menu\n";
    cout<<"3. Go to Main Menu..\n";
    cout<<"4. Exit\n";
    c = getch();
    if ( c == '4'){
        thanksLine();
    }
}

```

```

        exit(0);
    }else{
        return (c);
    }
}

int insertNode(int opr)
{
    system("cls");
    if ( opr == 1){
        cout<<"\n\nInsert At Beginning . \n\n";
        getPost();
        NODE *TEMP = new NODE(instragramid,UName, Uinstagram);
        if ( TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
            return 0;
        }
        if ( START == nullptr){
            END = START = TEMP;
        }
        else{
            TEMP->NEXT = START;
            START = TEMP;
        }
        countNode++;
        ICNT++;
    }
    else if ( opr == 2){
        cout<<"\n\nInsert At Post End . \n\n";
        getPost();
        NODE *TEMP = new NODE(instragramid,UName, Uinstagram);
    }
}

```

```

if ( TEMP == nullptr){
    cout<<"Insufficient Memory.. \n";
    return 0;
}
if ( START == nullptr){
    END = START = TEMP;
}
else{
    END->NEXT = TEMP;
    END = TEMP;
}
countNode++;
ICNT++;
}
else if ( opr == 3){
    cout<<"\n\nInsert Post At Given Position . \n\n";
    cout<<"Total Post "<<"\n\n";
    int pos;
    getPost();
    cout<<"Enter Position Between 0 to "<<countNode<<" -- ";
    cin>>pos;

    NODE *TEMP = new NODE(instragramid,UName, Uinstagram);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODE *TEMP1 = START;
    for ( i = 2 ; i < pos ; i++){

```

```

        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{
        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        if ( TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    countNode++;
    ICNT++;
}
char c = continueCheck();
if (c == '1')
    insertNode(opr);
else if ( c == '2')
    insertionOperation();
else if ( c == '3')
    MainMenu();
}

void showNode()
{
    cout<<"\n\ninstagram Post -- \n\n";
    cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"null \n\nPost is empty\n";
    }
}

```

```

    }
    else{
        while ( TEMP!= nullptr){
            cout<<"["<<TEMP->instragramid<< ", ";
            cout<<TEMP->UNAME<< ", ";
            cout<<TEMP->instagram<< "]" -> ";
            TEMP = TEMP->NEXT;
        }
        cout<<"Null";
    }
}

void countNodes()
{
    system("cls");
    cout<<"Count instagram Module --\n\n";
    cout<<"Total Number of Post -- "<<countNode<<" Post Available\n\n";
}

int insertionOperation()
{
    system("cls");
    cout<<"\nInsert Post..\n\n";
    cout<<"1. Insert Post At Beginning\n";
    cout<<"2. Insert Post At End\n";
    cout<<"3. Insert After Post\n";
    cout<<"4. Go to Main Menu\n";
    cout<<"5. Exit.\n";
    char c = getch();
    if ( c == '1'){
        insertNode(1);
    }
}

```

```

else if ( c == '2'){
    insertNode(2);
}
else if ( c == '3'){
    insertNode(3);
}
else if ( c == '4'){
    MainMenu();
}
else if ( c == '5'){
    thanksLine();
    exit(0);
}
}

int deleteNode(int opr)
{
    NODE *TEMP;
    if ( opr == 1){
        cout<<"\n\nDelete From Beginning . \n\n";
        if ( START == nullptr)
        {
            cout<<"Post is Empty.. \n\n";
            return 0;
        }
        else if ( START->NEXT == nullptr){
            TEMP = START;
            END = START = nullptr;
        }
        else{

```



```

        TEMP = START;
        START = START->NEXT;
    }
    countNode--;
    delete(TEMP);
    cout<<"\nPost Deleted Successfully\n\n";
}
else if ( opr == 2){
    cout<<"\n\nDelete From End . \n\n";
    if ( START == nullptr)
    {
        cout<<"Post is Empty.. \n\n";
        return 0;
    }
    else if ( START->NEXT == nullptr){
        TEMP = START;
        END = START = nullptr;
    }
    else{
        TEMP = START;
        int c = 1;
        while ( TEMP->NEXT != END)
        {
            cout<<c++<<endl;
            TEMP = TEMP->NEXT;
        }
        NODE *T1 = END;
        TEMP->NEXT = nullptr;
        END = TEMP;
        cout<<END->UNAME<<endl;
    }
}

```

```

        delete(T1);
    }
    countNode--;
    cout<<"\nPost Deleted Successfully\n\n";
}
else if ( opr == 3){
    cout<<"\n\nDelete From Given Position . \n\n";

    if ( START == nullptr)
    {
        cout<<"Post is Empty.. \n\n";
        return 0;
    }
    int pos;
    cout<<"Enter Position for Delete (1 - "<<countNode<<" ) - ";
    cin>>pos;
    if ( pos == 1 && START == END)
    {
        TEMP = START;
        START = END = nullptr;
    }
    else{
        TEMP = START;
        for (int i = 2 ; i < pos ; i++)
        {
            TEMP = TEMP->NEXT;
        }
        if (TEMP->NEXT == END){
            NODE *T1 = END;
            TEMP->NEXT = nullptr;

```

```

        END = TEMP;
    }
    else{
        NODE *T1 = TEMP->NEXT;
        TEMP->NEXT = T1->NEXT;
        delete(T1);
    }
}
countNode--;
cout<<"\nPost Deleted Successfully\n\n";
}
char c = continueCheck();
if (c == '1')
    deleteNode(opr);
else if ( c == '2')
    deleteOperation();
else if ( c == '3')
    MainMenu();
}
int deleteOperation()
{
    system("cls");
    cout<<"\nDelete Post..\n\n";
    cout<<"1. Delete Post From Beginning\n";
    cout<<"2. Delete Post From End\n";
    cout<<"3. Delete Post From Given Position\n";
    cout<<"4. Go to Main Menu\n";
    cout<<"5. Exit.\n";
    char c = getch();
    if ( c == '1'){

```

```

        deleteNode(1);
    }
    else if ( c == '2'){
        deleteNode(2);
    }
    else if ( c == '3'){
        deleteNode(3);
    }
    else if ( c == '4'){
        MainMenu();
    }
    else if ( c == '5'){
        thanksLine();
        exit(0);
    }
}
/*search module start*/

```

```

int searchinstagramInNode()
{
    cout<<"\n\nSearch Instagram Post in Nodes -- \n\n";
    cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
        cout<<"null \n\nlist is empty\n";
        return 0;
    }
    else{
        cout<<"\n\nEnter User Name To Find instagrams -- ";
        string uname;
    }
}

```

```

int counter = 0;
getline(cin,uname);
while ( TEMP!= nullptr){
    if (TEMP->UNAME == uname)
    {
        cout<<"\n["<<TEMP->instragramid<<", ";
        cout<<TEMP->UNAME<<", ";
        cout<<TEMP->instagram<<"]\n";
        counter++;
    }
    TEMP = TEMP->NEXT;
}
if ( counter == 0){
    cout<<"\nNo instagram Found..\n\n";
}
}
char c;
cout<<"\n\nAre you .. \n";
cout<<"1. Continue with Same\n";
cout<<"2. Go to Main Menu..\n";
cout<<"3. Exit\n";
c = getch();
if (c == '1')
    searchinstagramInNode();
else if ( c == '2')
    MainMenu();
else if ( c == '3'){
    thanksLine();
    exit(0);
}

```

```

}

/*Search Module End*/

int MainMenu()
{
    system("cls");
    cout<<"===== Instragram Post Feed =====\n\n";
    cout<<"1. Add Instagram Details \n";
    cout<<"2. Show Instagram Post\n";
    cout<<"3. Delete Instagram Post\n";
    cout<<"4. Search Instagram Post\n";
    cout<<"5. Count Instagram Post\n";
    cout<<"6. Exit\n";
    cout<<"Choose One of them -- ";

    char c = getch();
    if ( c == '1'){
        insertionOperation();
    }
    else if ( c == '2'){
        showNode();
    }
    else if ( c == '3'){
        deleteOperation();
    }
    else if ( c == '4'){
        searchinstagramInNode();
    }
    else if ( c == '5'){
        countNodes();
    }
}

```

```

    }
    else if ( c == '6'){
        thanksLine();
        exit(0);
    }
    cout<<"\n\nDo you want to continue .. press 1 else any key..";
    c = getch();
    if ( c == '1'){
        MainMenu();
    }
}

void thanksLine()
{
    cout<<"\n\nThanks for Using My APP\n\n";
}

};

int main()
{
    LinkList l;
    l.MainMenu();
    cout<<"\n\nThanks for Using My APP\n\n";
}

```

OUTPUT

```
===== Instragram Post Feed =====
```

1. Add instagram Details
 2. Show instagram Post
 3. Delete instagram Post
 4. Search instagram Post
 5. Count instagram Post
 6. Exit
- Choose One of them --

```
Insert Post..
```

1. Insert Post At Beginning
 2. Insert Post At End
 3. Insert After Post
 4. Go to Main Menu
 5. Exit.
- |

```
Insert At Beginning .
```

```
Enter Post id -- instagram Post1001  
Enter Username -- @jaishree19  
Enter Caption -- sweet
```

```
Are you..
```

1. Continue with Same
2. Go to Main Previous Menu
3. Go to Main Menu..
4. Exit


```
===== Instragram Post Feed =====
```

1. Add instagram Details
2. Show instagram Post
3. Delete instagram Post
4. Search instagram Post
5. Count instagram Post
6. Exit

Choose One of them --

instagram Post --

* Head -> [instagram Post1001, @jaishree19, sweet] -> Null

Do you want to continue .. press 1 else any key..|

Delete Post..

1. Delete Post From Beginning
2. Delete Post From End
3. Delete Post From Given Position
4. Go to Main Menu
5. Exit.

|

Delete Post..

1. Delete Post From Beginning
2. Delete Post From End
3. Delete Post From Given Position
4. Go to Main Menu
5. Exit.

Delete From Beginning .

Post Deleted Successfully

Are you..

1. Continue with Same
2. Go to Main Previous Menu
3. Go to Main Menu..
4. Exit

|

Search Instagram Post in Nodes --

* Head ->

Enter User Name To Find instagrams -- @jaishree19

[instagram Post1001, @jaishree19, sweet]

Are you ..

1. Continue with Same
2. Go to Main Menu..
3. Exit

|

```
Count instagram Module --
```

```
Total Number of Post -- 1 Post Available
```

```
Do you want to continue .. press 1 else any key..|
```

Dubbly LinkList

```
#include<iostream>
#include<conio.h>
#include<string>
using namespace std;
class NODED
{
public:
    NODED *PREV;
    string ID;
    string UNAME;
    string Instagram ;
    NODED *NEXT;
    NODED(string x, string y, string z)
    {
        PREV = nullptr;
        ID = x;
        UNAME = y;
        Instagram = z;
        NEXT = nullptr;
    }
};
class DLinkedList
{
public:
    string instragramid;
    int ICNT;
    string UName, UInstagram ;
    NODED *START,*END;
    int countNode;
```

```

DLinkedList(){
    ICNT = 1001;
    countNode = 0;
    START = nullptr;
    END = nullptr;
}

void getPost()
{
    instragramid = "Instagram Post"+ to_string(ICNT);
    cout<<"\nEnter Instragram Post id -- "<<instragramid<<endl;
    //getline(cin,instragramid);
    cout<<"Enter Username -- ";
    //cin.ignore();
    getline(cin,UName);
    cout<<"Enter Post Caption -- ";
    getline(cin,UInstagram );
}

char continueCheck()
{
    char c;
    cout<<"\n\nAre you.. \n";
    cout<<"1. Continue with Same\n";
    cout<<"2. Go to Main Previous Menu\n";
    cout<<"3. Go to Main Menu..\n";
    cout<<"4. Exit\n";
    c = getch();
    if ( c == '4'){
        thanksLine();
        exit(0);
    }else{

```

```

        return (c);
    }
}

int insertNode(int opr)
{
    system("cls");
    if ( opr == 1){
        cout<<"\n\nInsert At Beginning . \n\n";
        getPost();
        NODED *TEMP = new NODED(instragramid,UName, UInstagram);
        if ( TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
            return 0;
        }
        if ( START == nullptr){
            END = START = TEMP;
        }
        else{
            TEMP->NEXT = START;
            START->PREV = TEMP;
            START = TEMP;
        }
        cout<<END->UNAME;
        countNode++;
        ICNT++;
    }
    else if ( opr == 2){
        cout<<"\n\nInsert At End . \n\n";
        getPost();
        NODED *TEMP = new NODED(instragramid,UName, UInstagram );
    }
}

```

```

if ( TEMP == nullptr){
    cout<<"Insufficient Memory.. \n";
    return 0;
}
if ( START == nullptr){
    END = START = TEMP;
}
else{
    END->NEXT = TEMP;
    TEMP->PREV = END;
    END = TEMP;
}
countNode++;
ICNT++;
}
else if ( opr == 3){
    cout<<"\n\nInsert At Given Position . \n\n";
    cout<<"Total Post -- "<<countNode<<"\n\n";
    int pos;
    getPost();
    cout<<"Enter Position Between 0 to "<<countNode<<" -- ";
    cin>>pos;

    NODED *TEMP = new NODED(instragramid,UName, UInstagram);
    if ( TEMP == nullptr){
        cout<<"Insufficient Memory.. \n";
        return 0;
    }
    int i;
    NODED *TEMP1 = START;

```

```

    for ( i = 2 ; i < pos ; i++){
        TEMP1 = TEMP1->NEXT;
    }
    if ( START == nullptr){
        END = START = TEMP;
    }
    else{

        TEMP->NEXT = TEMP1->NEXT;
        TEMP1->NEXT = TEMP;
        TEMP->PREV = TEMP1;
        TEMP->NEXT->PREV=TEMP;
        if ( TEMP1 == END){
            END = TEMP1->NEXT;
        }
    }
    countNode++;
    ICNT++;
}
char c = continueCheck();
if (c == '1')
    insertNode(opr);
else if ( c == '2')
    insertionOperation();
else if ( c == '3')
    MainMenu();
}
int showNode()
{
    cout<<"\n\nDoubly Link List Show Post -- \n\n";

```



```

cout<<"* Head -> ";
if ( START == nullptr){
    cout<<"null \n\nlist is empty\n";
    return 0;
}
cout<<"\n\n1.Show Forward Post..\n";
cout<<"2. Show Backward Post\n";
cout<<"3. Goto to Main Menu..\n";
cout<<"4. Exit\n";
char c = getch();
if ( c == '1'){
    cout<<"\n\nShow Forward Post \n\n";
    cout<<"null <- ";
    NODED *TEMP = START;
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<" ";
        cout<<TEMP->UNAME<<" ";
        cout<<TEMP->Instagram<<"] -> ";
        TEMP = TEMP->NEXT;
    }
    cout<<"Null\n\n";
}
else if ( c == '2'){
    cout<<"\n\nShow Backward Post\n\n";
    cout<<"null <- ";
    NODED *TEMP = END;
    while ( TEMP!= nullptr){
        cout<<"["<<TEMP->ID<<" ";
        cout<<TEMP->UNAME<<" ";
        cout<<TEMP->Instagram <<"] -> ";
    }
}

```

```

        TEMP = TEMP->PREV;
    }
    cout<<"Null\n\n";
}
else if ( c == '3'){
    MainMenu();
}
else if (c == '4'){
    exit(0);
}

cout<<"\n\nAre you .. \n";
cout<<"1. Continue with Same\n";
cout<<"2. Go to Main Menu..\n";
cout<<"3. Exit\n";
c = getch();
if (c == '1')
    showNode();
else if ( c == '2')
    MainMenu();
else if ( c == '3'){
    thanksLine();
    exit(0);
}

}

void countNodes()
{
    system("cls");
    cout<<"Count Post Module --\n\n";

```

```

        cout<<"Total Number of Post -- "<<countNode<<" Post Available\n\n"
; }

int insertionOperation()
{
    system("cls");
    cout<<"\nInsert Instragram Detail..\n\n";
    cout<<"1. Insert Post At Beginning\n";
    cout<<"2. Insert Post At End\n";
    cout<<"3. Insert After Post\n";
    cout<<"4. Go to Main Menu\n";
    cout<<"5. Exit.\n";
    char c = getch();
    if ( c == '1'){
        insertNode(1);
    }
    else if ( c == '2'){
        insertNode(2);
    }
    else if ( c == '3'){
        insertNode(3);
    }
    else if ( c == '4'){
        MainMenu();
    }
    else if ( c == '5'){
        thanksLine();
        exit(0);
    }
}

```

```

int deleteNode(int opr)
{
    NODED *TEMP;
    if ( opr == 1){
        cout<<"\n\nDelete From Beginning . \n\n";
        if ( START == nullptr)
        {
            cout<<"Post is Empty.. \n\n";
            return 0;
        }
        else if ( START->NEXT == nullptr){
            TEMP = START;
            END = START = nullptr;
        }
        else{
            TEMP = START;
            START = START->NEXT;
            START->PREV=nullptr;
        }
        countNode--;
        delete(TEMP);
        cout<<"\nPost Deleted Successfully\n\n";
    }
    else if ( opr == 2){
        cout<<"\n\nDelete From End . \n\n";
        if ( START == nullptr)
        {
            cout<<"Post is Empty.. \n\n";
            return 0;
        }
    }
}

```

```

else if ( START->NEXT == nullptr){
    TEMP = START;
    END = START = nullptr;
}
else{
    NODED *TEMP = END;
    END = END->PREV;
    END->NEXT = nullptr;
    delete(TEMP);
}
countNode--;
cout<<"\nPost Deleted Successfully\n\n";
}
else if ( opr == 3){
    cout<<"\n\nDelete From Given Position . \n\n";

    if ( START == nullptr)
    {
        cout<<"Post is Empty.. \n\n";
        return 0;
    }
    int pos;
    cout<<"Enter Position for Delete (1 - "<<countNode<<") - ";
    cin>>pos;
    if ( pos == 1 && START == END)
    {
        TEMP = START;
        START = END = nullptr;
    }
    else{

```

```

    TEMP = START;
    for (int i = 1 ; i < pos ; i++)
    {
        TEMP = TEMP->NEXT;
    }
    if (TEMP == END){
        TEMP->PREV->NEXT = nullptr;
        END = TEMP->PREV;
        delete(TEMP);
    }
    else{
        TEMP->PREV->NEXT = TEMP->NEXT;
        TEMP->NEXT->PREV = TEMP->PREV;
        delete(TEMP);
    }
}
countNode--;
cout<<"\nPost Deleted Successfully\n\n";
}
char c = continueCheck();
if (c == '1')
    deleteNode(opr);
else if ( c == '2')
    deleteOperation();
else if ( c == '3')
    MainMenu();
}
int deleteOperation()
{
    system("cls");

```

```

cout<<"\nDeletion Post..\n\n";
cout<<"1. Delete Post From Beginning\n";
cout<<"2. Delete Post From End\n";
cout<<"3. Delete Post From Given Position\n";
cout<<"4. Go to Main Menu\n";
cout<<"5. Exit.\n";
char c = getch();
if ( c == '1'){
    deleteNode(1);
}
else if ( c == '2'){
    deleteNode(2);
}
else if ( c == '3'){
    deleteNode(3);
}
else if ( c == '4'){
    MainMenu();
}
else if ( c == '5'){
    thanksLine();
    exit(0);
}
}
/*search module start*/

int searchInstagramInNode()
{
    cout<<"\n\nSearch Instagram Post in Nodes -- \n\n";
    cout<<"* Head -> ";

```

```

NODED *TEMP = START;
if ( START == nullptr){
    cout<<"null \n\nlist is empty\n";
    return 0;
}
else{
    cout<<"\n\nEnter User Name To Find Instagram Posts -- ";
    string uname;
    int counter = 0;
    getline(cin,uname);
    while ( TEMP!= nullptr){
        if (TEMP->UNAME == uname)
        {
            cout<<"\n["<<TEMP->ID<<" ";
            cout<<TEMP->UNAME<<" ";
            cout<<TEMP->Instagram <<"]\n";
            counter++;
        }
        TEMP = TEMP->NEXT;
    }
    if ( counter == 0){
        cout<<"\nNo Instagram Post Found..\n\n";
    }
}
char c;
cout<<"\n\nAre you .. \n";
cout<<"1. Continue with Same\n";
cout<<"2. Go to Main Menu..\n";
cout<<"3. Exit\n";
c = getch();

```



```

        if (c == '1')
            searchInstagramInNode();
        else if ( c == '2')
            MainMenu();
        else if ( c == '3'){
            thanksLine();
            exit(0);
        }
    }
}

/*Search Module End*/
int MainMenu()
{
    system("cls");
    cout<<"===== Instragram Post Feed =====\n\n";
    cout<<"1. Insert Instragram Post\n";
    cout<<"2. Show Instragram Post\n";
    cout<<"3. Delete Post\n";
    cout<<"4. Search Post\n";
    cout<<"5. Count Post\n";
    cout<<"6. Exit\n";
    cout<<"Choose One of them -- ";
    /*l.insertNode(101,"Pooja","Hello Friends");
    l.insertNode(102,"Niharika","Hello Pooja");
    l.insertNode(103,"Deepak","Hello Tejswi");
    l.showNode();*/
    char c = getch();
    if ( c == '1'){
        insertionOperation();
    }
}

```

```

else if ( c == '2'){
    showNode();
}
else if ( c == '3'){
    deleteOperation();
}
else if ( c == '4'){
    searchInstagramInNode();
}
else if ( c == '5'){
    countNodes();
}
else if ( c == '6'){
    thanksLine();
    exit(0);
}
cout<<"\n\nDo you want to continue .. press 1 else any key..";
c = getch();
if ( c == '1'){
    MainMenu();
}
return 0;
}

void thanksLine()
{
    cout<<"\n\nThanks for Using My APP\n\n";
}

};

int main()

```

```

{
    DLinkedList l;
    l.MainMenu();
    cout<<"\n\nThanks for using My APP\n\n";
}

```

OUTPUT

```

===== Instragram Post Feed =====
1. Insert Instragram Post
2. Show Instragram Post
3. Delete Post
4. Search Post
5. Count Post
6. Exit
Choose One of them -- |

```

```

Insert Instragram Detail..
1. Insert Post At Beginning
2. Insert Post At End
3. Insert After Post
4. Go to Main Menu
5. Exit.

```

```

Insert At Beginning .

Enter Instragram Post id -- Instragram Post1001
Enter Username -- @Jaishree19
Enter Post Caption -- sweet
@Jaishree19

Are you..
1. Continue with Same
2. Go to Main Previous Menu
3. Go to Main Menu..
4. Exit

```

```
Doubly Link List Show Post --
```

```
* Head ->
```

- 1.Show Forward Post ..
2. Show Backward Post
3. Goto to Main Menu..
4. Exit

```
Show Forward Post
```

```
null <- [Instagram Post1001, @Jaishree19, sweet] -> Null
```

```
Are you ..
```

1. Continue with Same
2. Go to Main Menu..
3. Exit

```
Deletion Post..
```

1. Delete Post From Beginning
2. Delete Post From End
3. Delete Post From Given Position
4. Go to Main Menu
5. Exit.

Deletion Post..

1. Delete Post From Beginning
2. Delete Post From End
3. Delete Post From Given Position
4. Go to Main Menu
5. Exit.

Delete From Beginning .

Post Deleted Successfully

Are you..

1. Continue with Same
2. Go to Main Previous Menu
3. Go to Main Menu..
4. Exit

Search Instagram Post in Nodes --

* Head ->

Enter User Name To Find Instagram Posts -- @Jaishree19

[Instagram Post1001, @Jaishree19, sweet]

Are you ..

1. Continue with Same
2. Go to Main Menu..
3. Exit

Count Post Module --

Total Number of Post -- 1 Post Available

Do you want to continue .. press 1 else any key..

CODE OF MAIN MODULE

```
#include<iostream>
#include<conio.h>
#include"studentlink.cpp"
#include"dbll.cpp"
class MainLinkList
{
public:
    LinkList l;
    DLinkList dl;
    void MainLinkListModule()
    {
        system("cls");
        cout<<"----- Welcome Instagram Feed System Project -----\\n\\n";
        cout<<"1. Singly Link List \\n\\n";
        cout<<"2. Doubly Link List \\n\\n";
        cout<<"3. Exit \\n\\n";
        cout<<"Choose any one of them....";
        char c = getch();
        if ( c == '1'){
            l.MainMenu();
        }
        else if ( c == '2'){
            dl.MainMenu();
        }
        else if ( c == '3'){
            exit(0);
        }
    }
}
```

```

    }
};

int main()
{
    MainLinkList ll;

    ll.MainLinkListModule();

    cout<<"\n\nThank You for Using My App..\n\n";
    cout<<"Design & Coded By -- Jaishree Gahlot.. \n\n";

}

```

OUTPUT

```

----- Welcome Instragram Feed System Project -----
1. Singly Link List
2. Doubly Link List
3. Exit
Choose any one of them....

```

```

===== Instragram Post Feed =====
1. Add instagram Details
2. Show instagram Post
3. Delete instagram Post
4. Search instagram Post
5. Count instagram Post
6. Exit
Choose One of them --

```

Scan & Check my Project on Git Hub



Thank You
Dezyne École College
BCA 2nd Semester
Jaishree Gahlot